SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1735**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 2 **Duration : 1 Hour**
Total Marks:20

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

*I **(YAMUNA K/192511013)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
K. YAMUNA

Annexure A

CO1 Assessment: Constraint-Based Problem Solving

1. Hospital Shift Scheduling (Backtracking Search)

A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night).

Constraints:

i. D1 cannot work the Night shift.
ii. D2 must work before D3 (Morning < Afternoon < Night).
iii. Only one doctor per shift.
iv. D3 cannot work the Morning shift.
v. All doctors must be assigned exactly one shift.

Apply Backtracking Search to find a valid assignment, showing all intermediate steps and constraint checks. State the final valid schedule.

2. Robot Navigation in a 5×5 Grid (Heuristic Search)

A robot operates in a 5×5 grid environment. It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

i. Movement allowed: Up, Down, Left, Right.
ii. Cost of each move = 1.
iii. Cannot pass through obstacles.
iv. Use the Manhattan distance heuristic.

Using the above constraints and the Best-First Search algorithm, show step-by-step node expansion and priority queue updates, and determine the optimal path and its cost.

3. Autonomous Rescue Robot (Online Search in a Partially Observable Environment)

An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. It operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information. The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left and right.
- Some cells are blocked (obstacles) and cannot be traversed.
- The robot has limited sensing capability (it can only observe adjacent cells).
- Each movement costs 1, but entering a risky zone costs an additional 2.
- The robot must minimise total cost while ensuring safe navigation.
- The robot must update its knowledge dynamically (online search scenario).

Tasks:

a) Analyse all the given constraints and explain how they affect the robot's decision-making process.
b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search or an Online Search Agent). Clearly explain the steps involved.

c) Demonstrate how the robot applies the AI concepts of intelligent agents, rationality and environment type.
d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

4. Map Colouring of South Indian States (CSP with Heuristics)

Colour the map of the following states with three colours {Red, Green, Blue} so that no two neighbouring states have the same colour: Tamil Nadu (TN), Kerala (KL), Karnataka (KA), Andhra Pradesh (AP), Telangana (TG), Goa (GA) and Maharashtra (MH).

Neighbouring pairs: TN–KL, TN–KA, TN–AP, KL–KA, KA–AP, KA–TG, KA–GA, KA–MH, AP–TG, GA–MH, MH–TG.

i. Formulate the problem as a CSP: variables, domains and constraints. Draw the constraint graph.
ii. Use the Degree heuristic to choose the first variable, then the MRV (Minimum Remaining Values) heuristic for the remaining variables.
iii. Solve the problem using backtracking with forward checking, showing the domains after each assignment.
iv. State the final colouring and explain how the heuristics reduced backtracking.

5. 4×4 Sudoku (Constraint Propagation with AC-3)

Fill the grid so that every row, every column and every 2×2 box contains the digits 1–4 exactly once. Blank cells are shown as –.

–	2	–	–
3	–	–	2
–	–	4	–
4	–	–	1

i. Define the variables, domains and constraints (use the Alldiff constraint).
ii. Write the initial domain of every blank cell after removing values that conflict with the given digits.
iii. Apply arc consistency (AC-3) and show how the domains shrink.
iv. Complete the grid, using backtracking only if constraint propagation alone is not enough.

6. Cryptarithmetic Puzzle (CSP with Global Constraints)

Solve the puzzle below, where each letter stands for a different digit (0–9) and the leading letters T and F are not zero.

	T	W	O
+	T	W	O
F	O	U	R

i. Define the variables (including carry variables C1, C2, C3), their domains and all constraints.
ii. Write the column constraints as equations.
iii. Draw the constraint hypergraph.
iv. Solve the puzzle step by step using backtracking with constraint checks, and give one valid assignment.

7. Ambulance Routing During Floods (A* Search with a Road-Closure Constraint)

An ambulance must travel from the accident site S to the hospital G. Because of flooding, the road B–D is closed and cannot be used. The road costs (travel time in minutes) are:

Road	Cost	Road	Cost
S–A	3	C–D	2
S–B	2	C–E	3
A–C	4	D–G	2
B–C	1	E–G	4
B–D	2 (closed)		

Heuristic values (straight-line time to G): h(S)=6, h(A)=6, h(B)=4, h(C)=4, h(D)=2, h(E)=3, h(G)=0.

i. Apply A* search and show the open and closed lists, with g, h and f values, at each step.
ii. Give the route found and its total cost.
iii. Find the route and cost that A* would return if road B–D were open.
iv. Check whether the heuristic is admissible and consistent for the flooded network.

8. 4-Queens Problem (Local Search with Min-Conflicts)

Place 4 queens on a 4×4 board so that no two queens attack each other. Each column holds exactly one queen. The initial state has the queens in rows (1, 3, 4, 2) for columns 1 to 4 respectively, with row 1 at the top.

i. Formulate the problem as a CSP: variables, domains and constraints.
ii. Draw the initial board and count the number of conflicts for each queen.
iii. Apply the Min-Conflicts algorithm: at each step pick the leftmost queen in conflict and move it to the row with the fewest conflicts (break ties by choosing the lowest row number). Show the board and conflict counts after every move.
iv. State the final solution and explain why Min-Conflicts works well for large n-Queens problems.

9. Missionaries and Cannibals (State-Space Search under Safety Constraints)

Three missionaries and three cannibals are on the left bank of a river with a boat that carries at most two people. The boat cannot cross empty. On either bank, the missionaries must never be outnumbered by cannibals (if any missionaries are present).

i. Formulate the problem: state representation, initial state, actions, transition model, goal test and path cost.
ii. Identify which states violate the safety constraint and must be pruned.
iii. Draw the state space and solve the problem using Breadth-First Search.
iv. Give the minimum number of crossings and justify why BFS guarantees this minimum.

10. Sensorless Corridor Robot (Search with Partial Information)

A cleaning robot is placed in a corridor of four cells numbered 1 to 4 from left to right. It has no sensors, so it does not know which cell it starts in. The action Right moves it one cell to the right (it stays in cell 4 if already there), and Left moves it one cell to the left (it stays in cell 1 if already there). The robot must be certain that it is in cell 4.

i. Define the initial belief state and the size of the belief-state space.

ii. Show the belief state after each action in the sequence [Right, Right, Right].
iii. Prove that no shorter action sequence guarantees reaching cell 4.
iv. Suppose the robot gains a bump sensor that reports when it tries to move into a wall. Write a contingency plan that reaches cell 4 and compare it with the sensorless plan.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Hospital Shift Scheduling (Backtracking Search)

Aim
To assign three doctors (D1, D2 and D3) to three shifts using Backtracking Search while satisfying all the given constraints.

Problem Understanding
There are three shifts: Morning, Afternoon and Night. Each doctor must be assigned exactly one shift, and each shift must have exactly one doctor.

Constraints
1. D1 cannot work the Night shift.
2. D2 must work before D3.
3. Only one doctor can work each shift.
4. D3 cannot work the Morning shift.
5. All doctors must be assigned exactly one shift.

Algorithm
1. Start with an empty schedule.
2. Select a doctor and assign an available shift.
3. Check whether the assignment satisfies all constraints.
4. If the assignment violates a constraint, undo it and try another shift.
5. Continue until all doctors are assigned.
6. Stop when a valid schedule is found.

Step-by-Step Solution
Step 1: Assign D2 to Morning.
- D2 is assigned to Morning.
- This leaves Afternoon and Night for D1 and D3.

Step 2: Assign D3 to Afternoon.
- D2 works before D3.
- D3 is not assigned to Morning.
- The assignment satisfies both ordering and shift restrictions.

Step 3: Assign D1 to Night.
- This assignment violates the constraint that D1 cannot work the Night shift.
- Therefore, backtrack and try another assignment.

Step 4: Assign D1 to Afternoon and D3 to Night.
- D2 = Morning, D1 = Afternoon, D3 = Night.
- All three doctors have exactly one shift.
- Every shift has exactly one doctor.
- D1 is not assigned to Night.
- D3 is not assigned to Morning.
- D2 works before D3.

Final Valid Schedule

Shift	Doctor
Morning	D2
Afternoon	D1
Night	D3

Result
The hospital shift scheduling problem is solved using Backtracking Search. The final schedule satisfies all the constraints.

2. Robot Navigation in a 5×5 Grid (Heuristic Search)

Aim
To find a path from Start (S) to Goal (G) using Best-First Search and the Manhattan distance heuristic.

Problem Understanding
The robot moves through a 5×5 grid. It can move Up, Down, Left and Right. Each move costs 1, and obstacles cannot be crossed.

Heuristic Function
The Manhattan distance estimates the distance from a cell to the goal.
$$ h(n)=|x_n-x_G|+|y_n-y_G| $$
Where:

- (x_n,y_n) are the current cell coordinates.
- (x_G,y_G) are the goal coordinates.

Best-First Search selects the node with the smallest heuristic value $(h(n))$.

Algorithm

1. Place the start node in the priority queue.
2. Calculate the Manhattan distance for the start node.
3. Remove the node with the smallest heuristic value.
4. Expand its valid neighbouring cells.
5. Reject obstacles and already explored cells.
6. Add the remaining cells to the priority queue.
7. Repeat until the goal is reached.

Important Limitation in the Question
The assessment does not specify the exact locations of S, G and the obstacles in the 5×5 grid. Therefore, the exact node-expansion sequence, priority-queue updates and route cost cannot be uniquely determined from the question as provided.

Illustrative Example
For demonstration only, assume:

- Start: (1,1)
- Goal: (5,5)
- No obstacles

The initial heuristic is:
$$ h(S)=|1-5|+|1-5|=8 $$
The robot can follow this path:
$$ (1,1)\rightarrow(2,1)\rightarrow(3,1) \rightarrow(4,1)\rightarrow(5,1) $$
$$ \rightarrow(5,2)\rightarrow(5,3)\rightarrow(5,4) \rightarrow(5,5) $$
The total path cost is:
$$ g(G)=8\times1=8 $$
This is an illustrative path, not the unique answer for the assessment's unspecified obstacle layout.

Result
Best-First Search uses the Manhattan distance to guide the robot toward the goal. To calculate the exact assessment answer, the grid diagram or obstacle coordinates are required.

3. Autonomous Rescue Robot (Online Search in a Partially Observable Environment)

Aim
To develop an online search approach for an autonomous rescue robot that must locate a survivor while avoiding obstacles and minimising travel cost.

A. Analysis of Constraints

1. Movement constraints: The robot can move only Up, Down, Left and Right. Diagonal movement is not allowed.
2. Obstacles: Blocked cells cannot be entered.
3. Limited sensing: The robot observes only adjacent cells and cannot see the complete environment.
4. Movement cost: Each normal movement costs 1.
5. Risky zones: Entering a risky zone adds an additional cost of 2. Therefore, the total cost of entering such a cell is 3.
6. Dynamic knowledge: The robot updates its map as it discovers new cells.
7. Safety: The robot should avoid blocked paths and prefer safer routes when comparing travel costs.

B. Proposed Solution Using an Online Search Agent
Uniform Cost Search (UCS) is suitable when movement costs are non-negative and the objective is to minimise total path cost. Since the robot has incomplete information, it can use UCS on its currently known map and replan whenever it discovers new information.
Algorithm

1. Initialise the robot at the starting position S.
2. Add S to the frontier with cost 0.
3. Sense the adjacent cells.
4. Identify blocked, safe and risky cells.
5. Select the known frontier node with the lowest accumulated cost.
6. Move towards that node using the safest currently known route.
7. Update the map after each observation.
8. If a newly discovered obstacle blocks the planned path, replan.
9. Continue until the survivor at G is reached.

Cost Calculation
The movement cost is calculated as:
$$\text{Total Cost}=\text{Normal Moves}+3(\text{Risky Cells Entered})$$
Here, the formula counts every risky-cell entry as costing 3 in total, including its normal movement cost of 1 and the additional risk cost of 2.
The robot compares the total cost of known routes and selects the lowest-cost feasible route.

C. Application of AI Concepts
Intelligent agent: The rescue robot senses its surroundings, processes information and chooses actions to reach the survivor.
Rationality: A rational robot selects actions that minimise expected travel cost while respecting safety constraints and available information.
Environment type:

- Partially observable — the robot cannot observe the entire building.
- Sequential — each action affects future decisions.
- Dynamic — new information may change the route.
- Discrete — rooms and movements are represented as discrete states.
- Stochastic or deterministic — depends on whether the environment contains uncertain outcomes; the question does not specify this.

D. Justification
Online search is suitable because the robot does not initially know the complete environment. It can discover obstacles, update its knowledge and replan dynamically. Uniform Cost Search is appropriate for minimising accumulated cost, including the extra cost of risky zones. Safety rules must also be enforced during movement.

Result
An online search agent using cost-based route planning enables the rescue robot to adapt to incomplete information and find a feasible, low-cost route to the survivor.

4. Map Colouring of South Indian States (CSP with Heuristics)

Aim
To colour seven states using three colours so that no two neighbouring states have the same colour.

A. CSP Formulation
Variables: TN, KL, KA, AP, TG, GA and MH.
Domains:
$$ D=\{\text{Red, Green, Blue}\} $$
Constraints: Each neighbouring pair must have different colours.
The neighbouring pairs are:
TN–KL, TN–KA, TN–AP, KL–KA, KA–AP, KA–TG, KA–GA, KA–MH, AP–TG, GA–MH and MH–TG.

B. Constraint Graph
Constraint Graph (Adjacency List)
TN → KL, KA, AP
KL → TN, KA
KA → TN, KL, AP, TG, GA, MH
AP → TN, KA, TG
TG → KA, AP, MH
GA → KA, MH
MH → KA, GA, TG
An edge between two variables means the states are neighbours and cannot share a colour.

C. Apply the Degree Heuristic
The Degree heuristic selects the variable with the highest number of neighbouring variables.

State	Degree
KA	6
TN	3
AP	3
TG	3
KL	2
GA	2
MH	3

First variable: KA, because it has the highest degree.
Assign:
$$ KA=\text{Red} $$
Forward checking removes Red from the domains of TN, KL, AP, TG, GA and MH.
Their remaining domains become:
$$ \{\text{Green, Blue}\} $$

D. MRV and Backtracking with Forward Checking

The MRV heuristic chooses a variable with the fewest remaining legal values. At this stage, all unassigned variables have two values, so use degree as the tie-breaker.

Step 1: Assign \(TN=\text{Green}\).

- KL becomes .
- AP becomes .
- KA is already Red.

Step 2: Assign \(KL=\text{Blue}\).

Its neighbouring states TN and KA have different colours.

Step 3: Assign \(AP=\text{Blue}\).

TN is Green and KA is Red, so the assignment is valid.

Step 4: Assign \(TG=\text{Green}\).

TG differs from KA (Red) and AP (Blue).

Step 5: Assign \(MH=\text{Blue}\).

MH differs from KA (Red) and TG (Green).

Step 6: Assign \(GA=\text{Green}\).

GA differs from KA (Red) and MH (Blue).

Domain Updates

Step	Assignment	Important remaining domains
1	KA = Red	All neighbours:
2	TN = Green	KL = ; AP =
3	KL = Blue	No conflict
4	AP = Blue	TG =
5	TG = Green	MH =
6	MH = Blue	GA =
7	GA = Green	All variables assigned

E. Final Colouring

State	Colour
Tamil Nadu (TN)	Green
Kerala (KL)	Blue
Karnataka (KA)	Red
Andhra Pradesh (AP)	Blue
Telangana (TG)	Green
Goa (GA)	Green
Maharashtra (MH)	Blue

Result

The CSP is solved using the Degree heuristic, MRV and forward checking. Every neighbouring pair has different colours. The heuristics reduce unnecessary assignments by prioritising highly connected variables and detecting domain restrictions early.

5. 4×4 Sudoku (Constraint Propagation with AC-3)

Aim
To solve a 4×4 Sudoku using Constraint Satisfaction Problem (CSP) formulation and arc consistency.

A. Variables, Domains and Constraints
Variables: Each cell is represented as X_{ij}, where i is the row and j is the column.
Domains: Every blank cell has an initial domain drawn from:
$$ D=\{1,2,3,4\} $$
Constraints:

1. Every row must contain 1, 2, 3 and 4 exactly once.
2. Every column must contain 1, 2, 3 and 4 exactly once.
3. Every 2×2 box must contain 1, 2, 3 and 4 exactly once.

These are represented using the AllDiff constraint for each row, column and 2×2 box.

B. Problem in the Provided Grid
The uploaded assessment shows the following five lines of cell values:
As supplied, the values form five rows of four cells, not a 4×4 Sudoku.
This is an inconsistency in the question. A 4×4 Sudoku requires exactly four rows and four columns. Therefore, the initial domains and final solution cannot be determined reliably until the intended grid is confirmed.

C. AC-3 Procedure
For a correctly specified grid, AC-3 works as follows:

1. Create arcs between cells that share a row, column or 2×2 box.
2. Place these arcs in a queue.
3. Remove any candidate value from a cell's domain if it has no compatible value in a neighbouring cell's domain.
4. If a domain changes, add the relevant neighbouring arcs back to the queue.
5. Continue until the queue is empty or a domain becomes empty.
6. If all cells are not assigned, use backtracking and repeat constraint propagation.

For example, if a row already contains 2, then 2 is removed from the domains of its blank cells.

Result
The CSP formulation and AC-3 method are defined. A valid completed grid cannot be given without resolving the five-row versus four-row inconsistency in the assessment.

6. Cryptarithmetic Puzzle (CSP with Global Constraints)

Aim
To solve the cryptarithmetic puzzle TWO + TWO = FOUR using CSP and backtracking.

A. Variables and Domains
Letter variables:
$$ T,W,O,F,U,R\in\{0,1,\ldots,9\} $$
Carry variables:
$$ C_1,C_2,C_3\in\{0,1\} $$
Constraints:

- Each letter represents a different digit.
- $T\ne0$ and $F\ne0$.
- All arithmetic columns must be correct.
- Carry variables must satisfy the column equations.

B. Column Equations
Units column
$$ O+O=R+10C_1 $$
Tens column
$$ W+W+C_1=U+10C_2 $$
Hundreds column
$$ T+T+C_2=O+10C_3 $$
Thousands column
$$ C_3=F $$

C. Constraint Hypergraph
The variables in each column are linked by a constraint:

- Units: $\{O,R,C_1\}$
- Tens: $\{W,C_1,U,C_2\}$
- Hundreds: $\{T,C_2,O,C_3\}$
- Thousands: $\{C_3,F\}$
- Global constraint: all letter variables are different.

D. Backtracking Solution
We try digit assignments and reject any assignment that violates uniqueness or the column equations.
Step 1: Choose $T=7$, $W=3$, $O=4$.
This gives:
$$ 734+734=1468 $$
Step 2: Check the units column.
$$ 4+4=8 $$
Therefore, $R=8$ and $C_1=0$.
Step 3: Check the tens column.
$$ 3+3+0=6 $$
Therefore, $U=6$ and $C_2=0$.
Step 4: Check the hundreds column.
$$ 7+7+0=14 $$
Therefore, $O=4$ and $C_3=1$.
Step 5: Check the thousands column.
$$ F=C_3=1 $$
All six letters have distinct digits: 7, 3, 4, 1, 6 and 8.

Final Assignment

Variable	Digit
T	7
W	3
O	4
F	1
U	6
R	8
C_1	0
C_2	0
C_3	1

Verification
734 + 734 = 1468
Valid solution

Result
The puzzle is solved using backtracking, carry constraints and the global AllDiff constraint. The assignment satisfies all arithmetic and digit-uniqueness requirements.

7. Ambulance Routing During Floods (A* Search)

Aim
To find the minimum-cost route from the accident site S to the hospital G using A* search while accounting for a closed road.

A. Given Information
Road B–D is closed in the flooded network.
The A* evaluation function is:
$$ f(n)=g(n)+h(n) $$
Where:

- $g(n)$ = actual travel cost from S to node n.
- $h(n)$ = estimated remaining cost to G.
- $f(n)$ = estimated total route cost through node n.

B. A* Search with B–D Closed
Initial node:
$$ g(S)=0,\quad h(S)=6,\quad f(S)=6 $$
The open list is prioritised by increasing f.

Step	Expanded node	Open list after expansion (node: g, h, f)	Closed list
1	S	B: (2,4,6), A: (3,6,9)	S
2	B	C: (3,4,7), A: (3,6,9)	S, B
3	C	D: (5,2,7), A: (3,6,9), E: (6,3,9)	S, B, C
4	D	G: (7,0,7), A: (3,6,9), E: (6,3,9)	S, B, C, D
5	G	Goal reached	S, B, C, D, G

Road B–D is excluded because it is closed. The table uses the usual convention of selecting the lowest-f node first.

C. Route and Total Cost
The selected route is:
$$ S\rightarrow B\rightarrow C\rightarrow D\rightarrow G $$
Total cost:
$$ g(G)=2+1+2+2=\boxed{7\text{ minutes}} $$

D. If Road B–D Is Open
If B–D is available, the route through it has cost:
$$ S\rightarrow B\rightarrow D\rightarrow G $$
$$ g(G)=2+2+2=\boxed{6\text{ minutes}} $$
This is cheaper than the route through C.

E. Admissibility and Consistency
A heuristic is admissible if it never overestimates the actual minimum remaining cost.
For the flooded network, the true minimum remaining costs are:

Node	Given $h(n)$	True minimum cost	Admissible?
S	6	7	Yes
A	6	8	Yes

B	4	5	Yes
C	4	4	Yes
D	2	2	Yes
E	3	4	Yes
G	0	0	Yes

The heuristic is admissible for the flooded network.
A heuristic is consistent if:
$$ h(n)\le c(n,n')+h(n') $$
for every available road from n to n'.
Examples:

- S–B: $6\le2+4=6$
- B–C: $4\le1+4=5$
- C–D: $4\le2+2=4$
- D–G: $2\le2+0=2$

The remaining available edges also satisfy the consistency condition. Therefore, the heuristic is consistent for the flooded network.

Result
A* finds the route S–B–C–D–G with cost 7 minutes when B–D is closed. If B–D is open, it finds the cheaper route S–B–D–G with cost 6 minutes.

8. 4-Queens Problem (Local Search with Min-Conflicts)

Aim
To solve the 4-Queens problem using the Min-Conflicts algorithm so that no two queens attack each other.

A. CSP Formulation
Variables: Q_1,Q_2,Q_3,Q_4, where Q_i represents the row occupied by the queen in column i.
Domains:
$$ D(Q_i)=\{1,2,3,4\} $$
Constraints:

1. No two queens occupy the same row.
2. No two queens occupy the same diagonal.
3. Each column contains exactly one queen.

Two queens attack each other diagonally if:
$$ |Q_i-Q_j|=|i-j| $$

B. Initial State
The given initial rows are:
$$ (Q_1,Q_2,Q_3,Q_4)=(1,3,4,2) $$
Initial board:
Initial board: one queen in each column, at rows 1, 3, 4 and 2 respectively.
The only attacking pair is Q_2 and Q_3, which are on adjacent columns and adjacent rows.

Queen	Initial row	Number of conflicts
Q1	1	0
Q2	3	1
Q3	4	1
Q4	2	0

C. Apply Min-Conflicts
Rule: Select the leftmost queen in conflict. Move it to the row with the fewest conflicts, breaking ties by choosing the lowest row number.
Step 1: Select Q2.
Q2 is in row 3. Evaluate its possible rows against Q1, Q3 and Q4.

Candidate row for Q2	Number of conflicts
Row 1	1
Row 2	2
Row 3	1
Row 4	2

The minimum is 1, and the tie is between rows 1 and 3. The rule selects row 1.
New state:
$$ (1,1,4,2) $$
Now Q1 and Q2 conflict because they share the same row.
Step 2: Select Q1.
Evaluate possible rows for Q1 while Q2, Q3 and Q4 remain fixed.

Candidate row for Q1	Number of conflicts
Row 1	1
Row 2	3
Row 3	0
Row 4	1

The minimum is 0, so move Q1 to row 3.
New state:
$$ (3,1,4,2) $$

D. Final Board
Final solution: Q1 = 3, Q2 = 1, Q3 = 4, Q4 = 2.
Final state:
$$ \boxed{(3,1,4,2)} $$
There are zero conflicts. No queens share a row or diagonal.

E. Why Min-Conflicts Works Well
- It selects a variable that currently violates a constraint.
- It chooses a value that minimises conflicts with other variables.
- It avoids exploring every possible board configuration.
- It is efficient for large N-Queens problems in practice, although it does not guarantee success on every run.

Result
The 4-Queens problem is solved using Min-Conflicts in two moves, producing a valid arrangement with zero conflicts.

9. Missionaries and Cannibals (State-Space Search)

Aim
To transport three missionaries and three cannibals across a river safely using Breadth-First Search (BFS).

A. Problem Formulation
State representation:
$$ (M_L,C_L,B) $$
Where:
- M_L = missionaries on the left bank.
- C_L = cannibals on the left bank.
- B = boat position, L or R.

Initial state:
$$ (3,3,L) $$
Goal state:
$$ (0,0,R) $$
Actions: The boat can carry one or two people:
$$ \{M,C,MM,CC,MC\} $$
The boat cannot travel empty.
Transition model: A crossing transfers the selected passengers to the opposite bank.
Path cost: Each crossing costs 1.

B. Safety Constraints
On either bank, missionaries must not be outnumbered by cannibals whenever missionaries are present.
For the left bank:
$$ M_L=0\quad\text{or}\quad M_L\ge C_L $$
For the right bank:
$$ M_R=0\quad\text{or}\quad M_R\ge C_R $$
Where:
$$ M_R=3-M_L,\qquad C_R=3-C_L $$
Examples of invalid states:
- $(2,3,L)$: Two missionaries and three cannibals on the left bank.
- $(1,2,L)$: One missionary and two cannibals on the left bank.
- $(1,3,L)$: One missionary and three cannibals on the left bank.

Such states must be pruned because they violate the safety constraint.

C. BFS Solution
BFS explores states level by level, ensuring that the first goal found uses the fewest crossings.
The following is a valid minimum-crossing sequence. Passenger abbreviations: M = missionary, C = cannibal.

Crossing	Passengers	Left bank after crossing	Boat
0	Initial state	3M, 3C	L
1	CC →	3M, 1C	R
2	C ←	3M, 2C	L
3	CC →	3M, 0C	R
4	C ←	3M, 1C	L
5	MM →	1M, 1C	R
6	MC ←	2M, 2C	L
7	MM →	0M, 2C	R
8	C ←	0M, 3C	L
9	CC →	0M, 1C	R
10	C ←	0M, 2C	L

11	CC →	0M, 0C	R

All intermediate states satisfy the safety constraints.

D. State-Space Search
BFS maintains a queue of discovered states.

1. Begin with $(3,3,L)$.
2. Generate all legal passenger combinations.
3. Reject states that violate safety rules.
4. Reject states already visited.
5. Add valid new states to the queue.
6. Continue until $(0,0,R)$ is reached.
7. Trace back through the parent states to obtain the solution path.

E. Minimum Crossings and Justification
$$\boxed{\text{Minimum crossings}=11}$$
BFS guarantees the minimum number of crossings because every legal crossing has the same cost of 1. It explores all states reachable in k crossings before exploring states requiring $k+1$ crossings.

Result
All six people are transported safely across the river in 11 crossings using BFS.

10. Sensor less Corridor Robot (Search with Partial Information)

Aim
To determine the action sequence required for a robot with no sensors to reach cell 4 with certainty.

A. Initial Belief State
The corridor contains four cells numbered 1 to 4. Since the robot does not know its starting position, its initial belief state contains all possible positions.
$$\boxed{B_0=\{1,2,3,4\}}$$
A belief state is the set of all positions where the robot could currently be located.
Each of the four cells can independently be included or excluded from a belief state. Therefore, the total number of possible belief states is:
$$2^4=\boxed{16}$$
This includes the empty set mathematically, although it is not a possible physical location set for a functioning robot.

B. Belief-State Updates
The Right action moves the robot one cell right, except at cell 4, where it stays in place.
Initial state:
$$B_0=\{1,2,3,4\}$$
After the first Right action:

- 1 moves to 2.
- 2 moves to 3.
- 3 moves to 4.
- 4 stays at 4.

$$B_1=\{2,3,4\}$$
After the second Right action:
$$B_2=\{3,4\}$$
After the third Right action:
$$B_3=\{4\}$$

Summary

Stage	Action	Belief state
0	Initial	1234
1	Right	234
2	Right	34
3	Right	4

The robot is certain to be in cell 4 after three Right actions.

C. Proof That No Shorter Sequence Guarantees Cell 4

The robot could initially be in cell 1. From cell 1, it needs three Right actions to reach cell 4.

- Zero actions: the robot may still be in cell 1.
- One action: it can be in cell 2.
- Two actions: it can be in cell 3.
- Three Right actions: every possible starting position maps to cell 4.

Left actions cannot help the robot starting in cell 1 reach cell 4 faster.
Therefore, no action sequence shorter than three actions can guarantee reaching cell 4.
$$\boxed{\text{Minimum actions}=3}$$

D. Contingency Plan with a Bump Sensor

A bump sensor reports when the robot tries to move into a wall.
Contingency plan:

1. Repeatedly move Left until the sensor reports a bump.
2. The bump confirms that the robot is at cell 1.
3. Move Right three times.
4. The robot reaches cell 4.

The plan adapts to sensor feedback and identifies the left boundary before moving to the goal.

E. Comparison

Feature	Sensorless plan	Bump-sensor plan
Initial knowledge	Unknown position	Unknown position
Strategy	Three Right actions	Move Left until bump, then three Right actions
Guaranteed goal	Yes	Yes
Minimum guaranteed actions	3	May require more than 3
Position information	Belief-state tracking	Learns the left boundary

Result

The sensor less robot reaches cell 4 with certainty after three Right actions. The bump-sensor plan provides additional information but requires potentially more movements.